

P0406-r1: Intrusive Containers

Project: ISO/IEC JTC1/SC22/WG21

Audience: LEWG

Repository: [hfinkel/intrusive-containers-proposal](https://ericniebler.com/2016/05/24/intrusive-containers-proposal/)

Author: [Hal Finkel](#) (Argonne National Laboratory)

Table of Contents

1	Background
2	Examples
3	Intrusive containers
4	Intrusive forward_list
5	Intrusive list
6	Intrusive set
7	Intrusive multiset
8	Intrusive unordered_set
9	Intrusive unordered_multiset
10	Acknowledgments

Intrusive Containers for the C++ Standard Library

§ 1. Background

Intrusive containers, which are non-owning containers, are a well-known technique for high-performance object management. These have been available for many years as part of [Boost.Intrusive](#), and a subset of that library is proposed for the C++ standard library in this paper.

Specifically, please read [Intrusive and non-intrusive containers](#) for background on intrusive containers, their advantages, and their disadvantages.

Compared to Boost.Intrusive, proposed here is:

1. Only the "base hook" mechanism, not the "member hook" mechanism.
2. The "safe mode" features are not included (i.e. the "base hooks" are trivially constructible).
3. The "auto-unlink" feature is not included.
4. Node algorithms with custom NodeTraits are not included.
5. Containers with custom ValueTraits are not included.
6. Only intrusive container types with direct existing standard-library analogs (i.e. list, forward_list, set, multiset, unordered_set, and unordered_multiset) are included.
7. The container interface is not as rich: only those interfaces analogous to those on existing containers are included. Many more, however, can be marked as noexcept.

§ 2. Examples

1.) Example of using an `intrusive_forward_list` .

```
...
#include <intrusive_forward_list>

class Thing : public std::intrusive_forward_list_element<> {
    int i;

public:
    Thing(int i) : i(i) {}
    int order() { return i; } const;
};

...

std::vector<Thing> Things;
for (int i = 0; i < 10; ++i)
    Things.push_back(Thing(i));

std::intrusive_forward_list<Thing> ThingList;
for (auto &T : Things)
    ThingList.push_front(T);

for (auto &T : ThingList)
    std::cout << T.order() << "\n";
```

2.) Example of using a recursive `intrusive_list` .

```
...
#include <intrusive_list>

class Recursive : public std::intrusive_list_element<> {
public:
    std::intrusive_list<Recursive> children;
};

...

Recursive r1, r2;
std::intrusive_list<Recursive> RecursiveList;
RecursiveList.insert(RecursiveList.begin(), r1);

auto &ChildRecursiveList = RecursiveList.begin()->children;
ChildRecursiveList.insert(ChildRecursiveList.begin(), r2);

std::cout << RecursiveList.size() << "\n";
std::cout << r1.children.size() << "\n";
std::cout << r2.children.size() << "\n";

RecursiveList.clear();
r1.clear();
```

3.) Example of using multiple containers for the same objects simultaneously.

```

...
#include <intrusive_multiset>
#include <intrusive_unordered_set>

...

class name_tag;
class age_tag;

class Person : public std::intrusive_multiset_element<age_tag>,
               public std::intrusive_unordered_set<name_tag> {
    std::string name;
    int age;

public:
    Person(const std::string &n, int a) : name(n), age(a) {}

    int Age() { return age; } const;

    friend std::size_t hashName(const Person &P) {
        return std::hash(P.name);
    }

    friend bool isNameEqual(const Person &P1, const Person &P2) {
        return P1.name == P2.name;
    }

    friend bool isAgeLessThan(const Person &P1, const Person &P2) {
        return P1.age < P2.age;
    }
};

...

Person John("John", 34), Bob("Bob", 22);
typedef std::intrusive_multiset<Person, age_tag, isAgeLessThan> PeopleByAgeTy;
PeopleByAgeTy PeopleByAge;
std::intrusive_unordered_set<Person, name_tag, hashName, isNameEqual> PeopleByName;

PeopleByAge.insert(John); PeopleByAge.insert(Bob);
PeopleByName.insert(John); PeopleByName.insert(Bob);

...

std::size_t countPeers(const std::string &Name) {
    auto I = PeopleByName.find(Name);
    if (I == PeopleByName.end())
        return 0;

    // An important aspect of intrusive containers is that getting an iterator
    // from an object reference is a constant-time operation.
    Person &P = *I;
    PeopleByAgeTy::iterator J = P;

    std::size_t Count = 0;
    for (auto K = ++J; K != PeopleByAge.end() && K->Age() == P.Age();
         ++K) { ++Count; }
    for (auto K = J; K->Age() == P.Age(); --K) {
        ++Count;
        if (K == PeopleByAge.begin())
            break;
    }

    return Count;
}

```

§ 3. Intrusive containers

The intrusive containers proposed here correspond to existing standard-library containers. Each intrusive container's member functions are equivalent to the corresponding container functions, except that:

- None of them allocate memory, and correspondingly, more of them are adorned with `noexcept` (except for the unordered containers, for which the allocator might throw, and for the associative containers, where the comparison object might throw).
- Several functions take parameters by reference, instead of by const reference.
- Functions taking `InputIterator` parameters modify the objects made available by those parameters.

Each intrusive container's iterators have the same requirements as the corresponding container's iterators, with the additional requirement that the iterator type can be constructed from the object type in constant time.

§ 4. Intrusive forward_list

Header `<intrusive_forward_list>` synopsis

```
namespace std {
    struct default_intrusive_tag;

    template <class Tag = default_intrusive_tag>
    class intrusive_forward_list_element {
        // implementation defined.
    };

    template <class T, class Tag = default_intrusive_tag>
    class intrusive_forward_list {
    public:
        // types:
        typedef value_type& reference;
        typedef const value_type& const_reference;
        typedef implementation-defined iterator;
        typedef implementation-defined const_iterator;
        typedef implementation-defined size_type;
        typedef implementation-defined difference_type;
        typedef T value_type;

        // construct/copy/destroy:
        intrusive_forward_list() noexcept;
        template <class InputIterator>
            intrusive_forward_list(InputIterator first, InputIterator last);
        intrusive_forward_list(intrusive_forward_list&& x) noexcept;
        ~intrusive_forward_list() noexcept;
        intrusive_forward_list& operator=(intrusive_forward_list&& x) noexcept;
        template <class InputIterator>
            void assign(InputIterator first, InputIterator last);

        // iterators:
        iterator before_begin() noexcept;
        const_iterator before_begin() const noexcept;
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;

        const_iterator cbegin() const noexcept;
        const_iterator cbefore_begin() const noexcept;
        const_iterator cend() const noexcept;

        // capacity:
```

```

bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// element access:
reference front() noexcept;
const_reference front() const noexcept;

// modifiers:
void push_front(T& x) noexcept;
void pop_front() noexcept;
iterator insert_after(const_iterator position, T& x) noexcept;
template <class InputIterator>
    iterator insert_after(const_iterator position, InputIterator first, InputIterator las
t);
iterator erase_after(const_iterator position) noexcept;
iterator erase_after(const_iterator position, const_iterator last) noexcept;
void swap(intrusive_forward_list&) noexcept;
void clear() noexcept;

// list operations:
void splice_after(const_iterator position, intrusive_forward_list& x) noexcept;
void splice_after(const_iterator position, intrusive_forward_list&& x) noexcept;
void splice_after(const_iterator position, intrusive_forward_list& x,
    const_iterator i) noexcept;
void splice_after(const_iterator position, intrusive_forward_list&& x,
    const_iterator i) noexcept;
void splice_after(const_iterator position, intrusive_forward_list& x,
    const_iterator first, const_iterator last) noexcept;
void splice_after(const_iterator position, intrusive_forward_list&& x,
    const_iterator first, const_iterator last) noexcept;

void remove(const T& value);
template <class Predicate> void remove_if(Predicate pred);

void unique();
template <class BinaryPredicate>
void unique(BinaryPredicate binary_pred);

void merge(list& x);
void merge(list&& x);
template <class Compare> void merge(list& x, Compare comp);
template <class Compare> void merge(list&& x, Compare comp);

void sort();
template <class Compare> void sort(Compare comp);

void reverse() noexcept;
};

// Comparison operators
template <class T, class Tag>
bool operator==(const intrusive_forward_list<T,Tag>& x,
    const intrusive_forward_list<T,Tag>& y);
template <class T, class Tag>
bool operator< (const intrusive_forward_list<T,Tag>& x,
    const intrusive_forward_list<T,Tag>& y);
template <class T, class Tag>
bool operator!=(const intrusive_forward_list<T,Tag>& x,
    const intrusive_forward_list<T,Tag>& y);
template <class T, class Tag>
bool operator> (const intrusive_forward_list<T,Tag>& x,
    const intrusive_forward_list<T,Tag>& y);
template <class T, class Tag>
bool operator>=(const intrusive_forward_list<T,Tag>& x,
    const intrusive_forward_list<T,Tag>& y);
template <class T, class Tag>

```

```

bool operator<=(const intrusive_forward_list<T,Tag>& x,
               const intrusive_forward_list<T,Tag>& y);

// Specialized algorithms:
template <class T, class Tag>
void swap(intrusive_forward_list<T,Tag>& x,
         intrusive_forward_list<T,Tag>& y);
}

```

Note that the Boost.Intrusive name for this type is `slist` instead of `forward_list`.

§ 5. Intrusive list

Header `<intrusive_list>` synopsis

```

namespace std {
    struct default_intrusive_tag;

    template <class Tag = default_intrusive_tag>
    class intrusive_list_element {
        // implementation defined.
    };

    template <class T, class Tag = default_intrusive_tag>
    class intrusive_list {
    public:
        // types:
        typedef value_type& reference;
        typedef const value_type& const_reference;
        typedef implementation-defined iterator;
        typedef implementation-defined const_iterator;
        typedef implementation-defined size_type;
        typedef implementation-defined difference_type;
        typedef T value_type;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

        // construct/copy/destroy:
        intrusive_list() noexcept;
        template <class InputIterator>
            intrusive_list(InputIterator first, InputIterator last);
        intrusive_list(intrusive_list&& x) noexcept;
        ~intrusive_list() noexcept;
        intrusive_list& operator=(intrusive_list&& x) noexcept;
        template <class InputIterator>
            void assign(InputIterator first, InputIterator last);

        // iterators:
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;

        reverse_iterator rbegin() noexcept;
        const_reverse_iterator rbegin() const noexcept;
        reverse_iterator rend() noexcept;
        const_reverse_iterator rend() const noexcept;

        const_iterator cbegin() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        const_reverse_iterator crend() const noexcept;

        // capacity:
        bool empty() const noexcept;

```

```

size_type size() const noexcept;
size_type max_size() const noexcept;

// element access:
reference front() noexcept;
const_reference front() const noexcept;
reference back() noexcept;
const_reference back() const noexcept;

// modifiers:
void push_front(T& x) noexcept;
void push_back(T& x) noexcept;
void pop_back() noexcept;
iterator insert(const_iterator position, T& x) noexcept;
template <class InputIterator>
    iterator insert(const_iterator position, InputIterator first, InputIterator last);
iterator erase(const_iterator position) noexcept;
iterator erase(const_iterator position, const_iterator last) noexcept;
void swap(intrusive_list&) noexcept;
void clear() noexcept;

// list operations:
void splice(const_iterator position, intrusive_list& x) noexcept;
void splice(const_iterator position, intrusive_list&& x) noexcept;
void splice(const_iterator position, intrusive_list& x, const_iterator i) noexcept;
void splice(const_iterator position, intrusive_list&& x, const_iterator i) noexcept;
void splice(const_iterator position, intrusive_list& x,
            const_iterator first, const_iterator last) noexcept;
void splice(const_iterator position, intrusive_list&& x,
            const_iterator first, const_iterator last) noexcept;

void remove(const T& value);
template <class Predicate> void remove_if(Predicate pred);

void unique();
template <class BinaryPredicate>
void unique(BinaryPredicate binary_pred);

void merge(list& x);
void merge(list&& x);
template <class Compare> void merge(list& x, Compare comp);
template <class Compare> void merge(list&& x, Compare comp);

void sort();
template <class Compare> void sort(Compare comp);

void reverse() noexcept;
};

// Comparison operators
template <class T, class Tag>
bool operator==(const intrusive_list<T,Tag>& x,
               const intrusive_list<T,Tag>& y);
template <class T, class Tag>
bool operator< (const intrusive_list<T,Tag>& x,
              const intrusive_list<T,Tag>& y);
template <class T, class Tag>
bool operator!=(const intrusive_list<T,Tag>& x,
              const intrusive_list<T,Tag>& y);
template <class T, class Tag>
bool operator> (const intrusive_list<T,Tag>& x,
              const intrusive_list<T,Tag>& y);
template <class T, class Tag>
bool operator>=(const intrusive_list<T,Tag>& x,
              const intrusive_list<T,Tag>& y);
template <class T, class Tag>
bool operator<=(const intrusive_list<T,Tag>& x,

```

```

        const intrusive_list<T,Tag>& y);

// Specialized algorithms:
template <class T, class Tag>
void swap(intrusive_forward_list<T,Tag>& x,
          intrusive_forward_list<T,Tag>& y);
}

```

§ 6. Intrusive set

Header <intrusive_set> synopsis

```

namespace std {
    struct default_intrusive_tag;

    template <class Tag = default_intrusive_tag>
    class intrusive_set_element {
        // implementation defined.
    };

    template <class T, class Tag = default_intrusive_tag,
              class Compare = less<T>>
    class intrusive_set {
    public:
        // types:
        typedef T key_type;
        typedef T value_type;
        typedef Compare key_compare;
        typedef Compare value_compare;
        typedef value_type& reference;
        typedef const value_type& const_reference;
        typedef implementation-defined iterator;
        typedef implementation-defined const_iterator;
        typedef implementation-defined size_type;
        typedef implementation-defined difference_type;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

        // construct/copy/destroy:
        intrusive_set() : intrusive_set(Compare()) { }
        explicit intrusive_set(const Compare& comp);
        template <class InputIterator>
            intrusive_set(InputIterator first, InputIterator last,
                          const Compare& comp = Compare());
        intrusive_set(intrusive_set&& x) noexcept;
        ~intrusive_set() noexcept;
        intrusive_set& operator=(intrusive_set&& x) noexcept;
        template <class InputIterator>
            void assign(InputIterator first, InputIterator last);

        // iterators:
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;

        reverse_iterator rbegin() noexcept;
        const_reverse_iterator rbegin() const noexcept;
        reverse_iterator rend() noexcept;
        const_reverse_iterator rend() const noexcept;

        const_iterator cbegin() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        const_reverse_iterator crend() const noexcept;
    };
}

```

```

// capacity:
bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers:
pair<iterator,bool> insert(value_type& x);
iterator insert(const_iterator position, value_type& x);
template <class InputIterator>
    iterator insert(const_iterator position, InputIterator first, InputIterator last);
iterator erase(const_iterator position) noexcept;
size_type erase(key_type& x);
iterator erase(const_iterator position, const_iterator last) noexcept;
void swap(intrusive_set&) noexcept;
void clear() noexcept;

// observers:
key_compare key_comp() const;
value_compare value_comp() const;

// set operations:
iterator find(const key_type& x);
const_iterator find(const key_type& x) const;
template <class K> iterator find(const K& x);
template <class K> const_iterator find(const K& x) const;

size_type count(const key_type& x) const;
template <class K> size_type count(const K& x) const;

iterator lower_bound(const key_type& x);
const_iterator lower_bound(const key_type& x) const;
template <class K> iterator lower_bound(const K& x);
template <class K> const_iterator lower_bound(const K& x) const;

iterator upper_bound(const key_type& x);
const_iterator upper_bound(const key_type& x) const;
template <class K> iterator upper_bound(const K& x);
template <class K> const_iterator upper_bound(const K& x) const;

pair<iterator,iterator> equal_range(const key_type& x);
pair<const_iterator,const_iterator> equal_range(const key_type& x) const;
template <class K> pair<iterator, iterator> equal_range(const K& x);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

// Comparison operators
template <class T, class Tag, class Compare>
bool operator==(const intrusive_set<T,Tag,Compare>& x,
                const intrusive_set<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator< (const intrusive_set<T,Tag,Compare>& x,
                const intrusive_set<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator!=(const intrusive_set<T,Tag,Compare>& x,
                const intrusive_set<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator> (const intrusive_set<T,Tag,Compare>& x,
                const intrusive_set<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator>=(const intrusive_set<T,Tag,Compare>& x,
                const intrusive_set<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator<=(const intrusive_set<T,Tag,Compare>& x,
                const intrusive_set<T,Tag,Compare>& y);

// Specialized algorithms:

```

```

template <class T, class Tag, class Compare>
void swap(intrusive_forward_set<T,Tag,Compare>& x,
          intrusive_forward_set<T,Tag,Compare>& y);
}

```

§ 7. Intrusive multiset

Header `<intrusive_multiset>` synopsis

```

namespace std {
    struct default_intrusive_tag;

    template <class Tag = default_intrusive_tag>
    class intrusive_multiset_element {
        // implementation defined.
    };

    template <class T, class Tag = default_intrusive_tag,
              class Compare = less<T>>
    class intrusive_multiset {
    public:
        // types:
        typedef T key_type;
        typedef T value_type;
        typedef Compare key_compare;
        typedef Compare value_compare;
        typedef value_type& reference;
        typedef const value_type& const_reference;
        typedef implementation-defined iterator;
        typedef implementation-defined const_iterator;
        typedef implementation-defined size_type;
        typedef implementation-defined difference_type;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

        // construct/copy/destroy:
        intrusive_multiset() : intrusive_multiset(Compare()) { }
        explicit intrusive_multiset(const Compare& comp);
        template <class InputIterator>
            intrusive_multiset(InputIterator first, InputIterator last,
                               const Compare& comp = Compare());
        intrusive_multiset(intrusive_multiset&& x) noexcept;
        ~intrusive_multiset() noexcept;
        intrusive_multiset& operator=(intrusive_multiset&& x) noexcept;
        template <class InputIterator>
            void assign(InputIterator first, InputIterator last);

        // iterators:
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;

        reverse_iterator rbegin() noexcept;
        const_reverse_iterator rbegin() const noexcept;
        reverse_iterator rend() noexcept;
        const_reverse_iterator rend() const noexcept;

        const_iterator cbegin() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        const_reverse_iterator crend() const noexcept;

        // capacity:
        bool empty() const noexcept;

```

```

size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers:
pair<iterator,bool> insert(value_type& x);
iterator insert(const_iterator position, value_type& x);
template <class InputIterator>
    iterator insert(const_iterator position, InputIterator first, InputIterator last);
iterator erase(const_iterator position) noexcept;
size_type erase(key_type& x);
iterator erase(const_iterator position, const_iterator last) noexcept;
void swap(intrusive_multiset&) noexcept;
void clear() noexcept;

// observers:
key_compare key_comp() const;
value_compare value_comp() const;

// set operations:
iterator find(const key_type& x);
const_iterator find(const key_type& x) const;
template <class K> iterator find(const K& x);
template <class K> const_iterator find(const K& x) const;

size_type count(const key_type& x) const;
template <class K> size_type count(const K& x) const;

iterator lower_bound(const key_type& x);
const_iterator lower_bound(const key_type& x) const;
template <class K> iterator lower_bound(const K& x);
template <class K> const_iterator lower_bound(const K& x) const;

iterator upper_bound(const key_type& x);
const_iterator upper_bound(const key_type& x) const;
template <class K> iterator upper_bound(const K& x);
template <class K> const_iterator upper_bound(const K& x) const;

pair<iterator,iterator> equal_range(const key_type& x);
pair<const_iterator,const_iterator> equal_range(const key_type& x) const;
template <class K> pair<iterator, iterator> equal_range(const K& x);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

// Comparison operators
template <class T, class Tag, class Compare>
bool operator==(const intrusive_multiset<T,Tag,Compare>& x,
                const intrusive_multiset<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator< (const intrusive_multiset<T,Tag,Compare>& x,
                const intrusive_multiset<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator!=(const intrusive_multiset<T,Tag,Compare>& x,
                const intrusive_multiset<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator> (const intrusive_multiset<T,Tag,Compare>& x,
                const intrusive_multiset<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator>=(const intrusive_multiset<T,Tag,Compare>& x,
                const intrusive_multiset<T,Tag,Compare>& y);
template <class T, class Tag, class Compare>
bool operator<=(const intrusive_multiset<T,Tag,Compare>& x,
                const intrusive_multiset<T,Tag,Compare>& y);

// Specialized algorithms:
template <class T, class Tag, class Compare>
void swap(intrusive_forward_multiset<T,Tag,Compare>& x,

```

```

        intrusive_forward_multiset<T, Tag, Compare>& y);
    }

```

§ 8. Intrusive unordered_set

Header <intrusive_unordered_set> synopsis

```

namespace std {
    struct default_intrusive_tag;

    template <class Tag = default_intrusive_tag>
    class intrusive_unordered_set_element {
        // implementation defined.
    };

    template <class T, class Tag = default_intrusive_tag,
              class Hash = hash<T>,
              class Pred = std::equal_to<T>,
              class Allocator = std::allocator<T>>
    class intrusive_unordered_set {
    public:
        // types:
        typedef T key_type;
        typedef T value_type;
        typedef Hash hasher;
        typedef Pred key_equal;

        typedef Allocator allocator_type;

        typedef value_type& reference;
        typedef const value_type& const_reference;

        typedef implementation-defined iterator;
        typedef implementation-defined const_iterator;
        typedef implementation-defined local_iterator;
        typedef implementation-defined const_local_iterator;

        typedef implementation-defined size_type;
        typedef implementation-defined difference_type;

        // construct/copy/destroy:
        intrusive_unordered_set();
        explicit intrusive_unordered_set(const hasher& hf = hasher(),
                                       const key_equal& eql = key_equal(),
                                       const allocator_type& a = allocator_type());

        template <class InputIterator>
            intrusive_unordered_set(InputIterator first, InputIterator last,
                                   size_type n = implementation defined,
                                   const hasher& hf = hasher(),
                                   const key_equal& eql = key_equal(),
                                   const allocator_type& a = allocator_type());
        intrusive_unordered_set(intrusive_unordered_set&& x) noexcept;
        explicit intrusive_unordered_set(const Allocator&);
        intrusive_unordered_set(intrusive_unordered_set&& x, const Allocator&);
        intrusive_unordered_set(size_type n, const allocator_type& a)
            : intrusive_unordered_set(n, hasher(), key_equal(), a) { }
        intrusive_unordered_set(size_type n, const hasher& hf, const allocator_type& a)
            : intrusive_unordered_set(n, hf, key_equal(), a) { }
        template <class InputIterator>
            intrusive_unordered_set(InputIterator f, InputIterator l, size_type n,
                                   const allocator_type& a)
            : intrusive_unordered_set(f, l, n, hasher(), key_equal(), a) { }
        template <class InputIterator>
            intrusive_unordered_set(InputIterator f, InputIterator l, size_type n, const hasher& hf

```

```

        const allocator_type& a)
    : intrusive_unordered_set(f, l, n, hf, key_equal(), a) { }
~intrusive_unordered_set() noexcept;
intrusive_unordered_set& operator=(intrusive_unordered_set&& x) noexcept;
template <class InputIterator>
    void assign(InputIterator first, InputIterator last);

// iterators:
iterator begin() noexcept;
const_iterator begin() const noexcept;
iterator end() noexcept;
const_iterator end() const noexcept;

const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// capacity:
bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers:
pair<iterator, bool> insert(value_type& x);
iterator insert(const_iterator hint, value_type& x);
template <class InputIterator>
    iterator insert(InputIterator first, InputIterator last);
iterator erase(const_iterator position) noexcept;
size_type erase(key_type& x);
iterator erase(const_iterator position, const_iterator last) noexcept;
void swap(intrusive_unordered_set&) noexcept;
void clear() noexcept;

// observers:
hasher hash_function() const;
key_equal key_eq() const;

// lookup:
iterator find(const key_type& x);
const_iterator find(const key_type& x) const;

size_type count(const key_type& x) const;
template <class K> size_type count(const K& x) const;

pair<iterator, iterator> equal_range(const key_type& x);
pair<const_iterator, const_iterator> equal_range(const key_type& x) const;

// bucket interface:
size_type bucket_count() const noexcept;
size_type max_bucket_count() const noexcept;
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;
local_iterator begin(size_type n);
const_local_iterator begin(size_type n) const;
local_iterator end(size_type n);
const_local_iterator end(size_type n) const;
const_local_iterator cbegin(size_type n) const;
const_local_iterator cend(size_type n) const;

// hash policy:
float load_factor() const noexcept;
float max_load_factor() const noexcept;
void max_load_factor(float z);
void rehash(size_type n);
void reserve(size_type n);
};

// Comparison operators

```

```

template <class T, class Tag, class Hash, class Pred, class Alloc>
bool operator==(const intrusive_unordered_set<T,Tag,Hash,Pred,Alloc>& x,
                const intrusive_unordered_set<T,Tag,Hash,Pred,Alloc>& y);
template <class T, class Tag, class Hash, class Pred, class Alloc>
bool operator!=(const intrusive_unordered_set<T,Tag,Hash,Pred,Alloc>& x,
                const intrusive_unordered_set<T,Tag,Hash,Pred,Alloc>& y);

// Specialized algorithms:
template <class T, class Tag, class Hash, class Pred, class Alloc>
void swap(intrusive_forward_unordered_set<T,Tag,Hash,Pred,Alloc>& x,
          intrusive_forward_unordered_set<T,Tag,Hash,Pred,Alloc>& y);
}

```

Note that this interface differs from that provided by Boost.Intrusive because Boost.Intrusive uses only user-provided memory for the hash-bucket array, whereas this class takes an allocator and manages that memory itself.

§ 9. Intrusive unordered_multiset

Header <intrusive_unordered_multiset> synopsis

```

namespace std {
    struct default_intrusive_tag;

    template <class Tag = default_intrusive_tag>
    class intrusive_unordered_multiset_element {
        // implementation defined.
    };

    template <class T, class Tag = default_intrusive_tag,
              class Hash = hash<T>,
              class Pred = std::equal_to<T>,
              class Allocator = std::allocator<T>>
    class intrusive_unordered_multiset {
    public:
        // types:
        typedef T key_type;
        typedef T value_type;
        typedef Hash hasher;
        typedef Pred key_equal;

        typedef Allocator allocator_type;

        typedef value_type& reference;
        typedef const value_type& const_reference;

        typedef implementation-defined iterator;
        typedef implementation-defined const_iterator;
        typedef implementation-defined local_iterator;
        typedef implementation-defined const_local_iterator;

        typedef implementation-defined size_type;
        typedef implementation-defined difference_type;

        // construct/copy/destroy:
        intrusive_unordered_multiset();
        explicit intrusive_unordered_multiset(const hasher& hf = hasher(),
                                              const key_equal& eql = key_equal(),
                                              const allocator_type& a = allocator_type());

        template <class InputIterator>
        intrusive_unordered_multiset(InputIterator first, InputIterator last,
                                     size_type n = implementation defined,
                                     const hasher& hf = hasher(),
                                     const key_equal& eql = key_equal(),
                                     const allocator_type& a = allocator_type());
    };
}

```

```

intrusive_unordered_multiset(intrusive_unordered_multiset&& x) noexcept;
explicit intrusive_unordered_multiset(const Allocator&);
intrusive_unordered_multiset(intrusive_unordered_multiset&& x, const Allocator&);
intrusive_unordered_multiset(size_type n, const allocator_type& a)
    : intrusive_unordered_multiset(n, hasher(), key_equal(), a) { }
intrusive_unordered_multiset(size_type n, const hasher& hf, const allocator_type& a)
    : intrusive_unordered_multiset(n, hf, key_equal(), a) { }
template <class InputIterator>
intrusive_unordered_multiset(InputIterator f, InputIterator l, size_type n,
    const allocator_type& a)
    : intrusive_unordered_multiset(f, l, n, hasher(), key_equal(), a) { }
template <class InputIterator>
intrusive_unordered_multiset(InputIterator f, InputIterator l, size_type n,
    const hasher& hf,
    const allocator_type& a)
    : intrusive_unordered_multiset(f, l, n, hf, key_equal(), a) { }
~intrusive_unordered_multiset() noexcept;
intrusive_unordered_multiset& operator=(intrusive_unordered_multiset&& x) noexcept;
template <class InputIterator>
    void assign(InputIterator first, InputIterator last);

// iterators:
iterator begin() noexcept;
const_iterator begin() const noexcept;
iterator end() noexcept;
const_iterator end() const noexcept;

const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// capacity:
bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers:
pair<iterator, bool> insert(value_type& x);
iterator insert(const_iterator hint, value_type& x);
template <class InputIterator>
    iterator insert(InputIterator first, InputIterator last);
iterator erase(const_iterator position) noexcept;
size_type erase(key_type& x);
iterator erase(const_iterator position, const_iterator last) noexcept;
void swap(intrusive_unordered_multiset&) noexcept;
void clear() noexcept;

// observers:
hasher hash_function() const;
key_equal key_eq() const;

// lookup:
iterator find(const key_type& x);
const_iterator find(const key_type& x) const;

size_type count(const key_type& x) const;
template <class K> size_type count(const K& x) const;

pair<iterator, iterator> equal_range(const key_type& x);
pair<const_iterator, const_iterator> equal_range(const key_type& x) const;

// bucket interface:
size_type bucket_count() const noexcept;
size_type max_bucket_count() const noexcept;
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;
local_iterator begin(size_type n);
const_local_iterator begin(size_type n) const;

```

```

    local_iterator end(size_type n);
    const_local_iterator end(size_type n) const;
    const_local_iterator cbegin(size_type n) const;
    const_local_iterator cend(size_type n) const;

    // hash policy:
    float load_factor() const noexcept;
    float max_load_factor() const noexcept;
    void max_load_factor(float z);
    void rehash(size_type n);
    void reserve(size_type n);
};

// Comparison operators
template <class T, class Tag, class Hash, class Pred, class Alloc>
bool operator==(const intrusive_unordered_multiset<T,Tag,Hash,Pred,Alloc>& x,
                const intrusive_unordered_multiset<T,Tag,Hash,Pred,Alloc>& y);
template <class T, class Tag, class Hash, class Pred, class Alloc>
bool operator!=(const intrusive_unordered_multiset<T,Tag,Hash,Pred,Alloc>& x,
                const intrusive_unordered_multiset<T,Tag,Hash,Pred,Alloc>& y);

// Specialized algorithms:
template <class T, class Tag, class Hash, class Pred, class Alloc>
void swap(intrusive_forward_unordered_multiset<T,Tag,Hash,Pred,Alloc>& x,
          intrusive_forward_unordered_multiset<T,Tag,Hash,Pred,Alloc>& y);
}

```

Note that this interface differs from that provided by Boost.Intrusive because Boost.Intrusive uses only user-provided memory for the hash-bucket array, whereas this class takes an allocator and manages that memory itself.

§ 10. Acknowledgments

The author thanks Ion Gaztanaga, Olaf Krzikalla, and other contributors to the Boost.Intrusive library. The author thanks Michael Wong and other SG14 participants for repeatedly stressing the desire for this feature. The author thanks Marshall Clow, Ville Voutilainen, and Jonathan Wakely for providing feedback.